

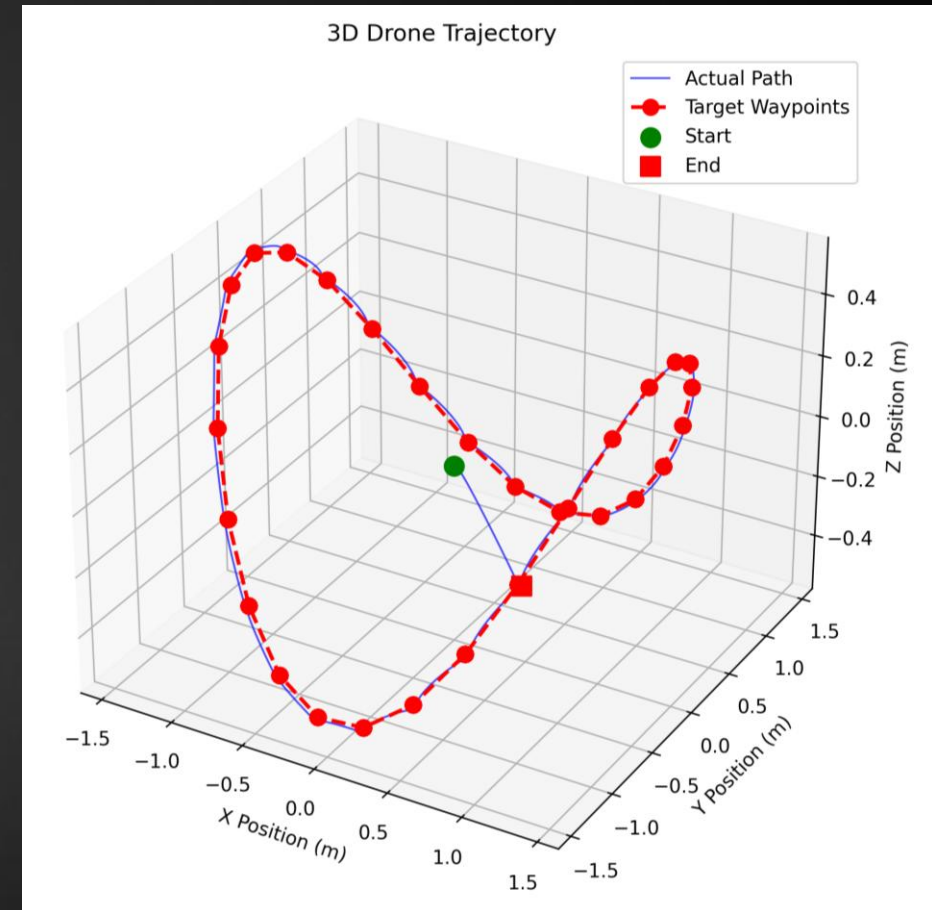
# Drone Simulation

GUANGZHI CHEN

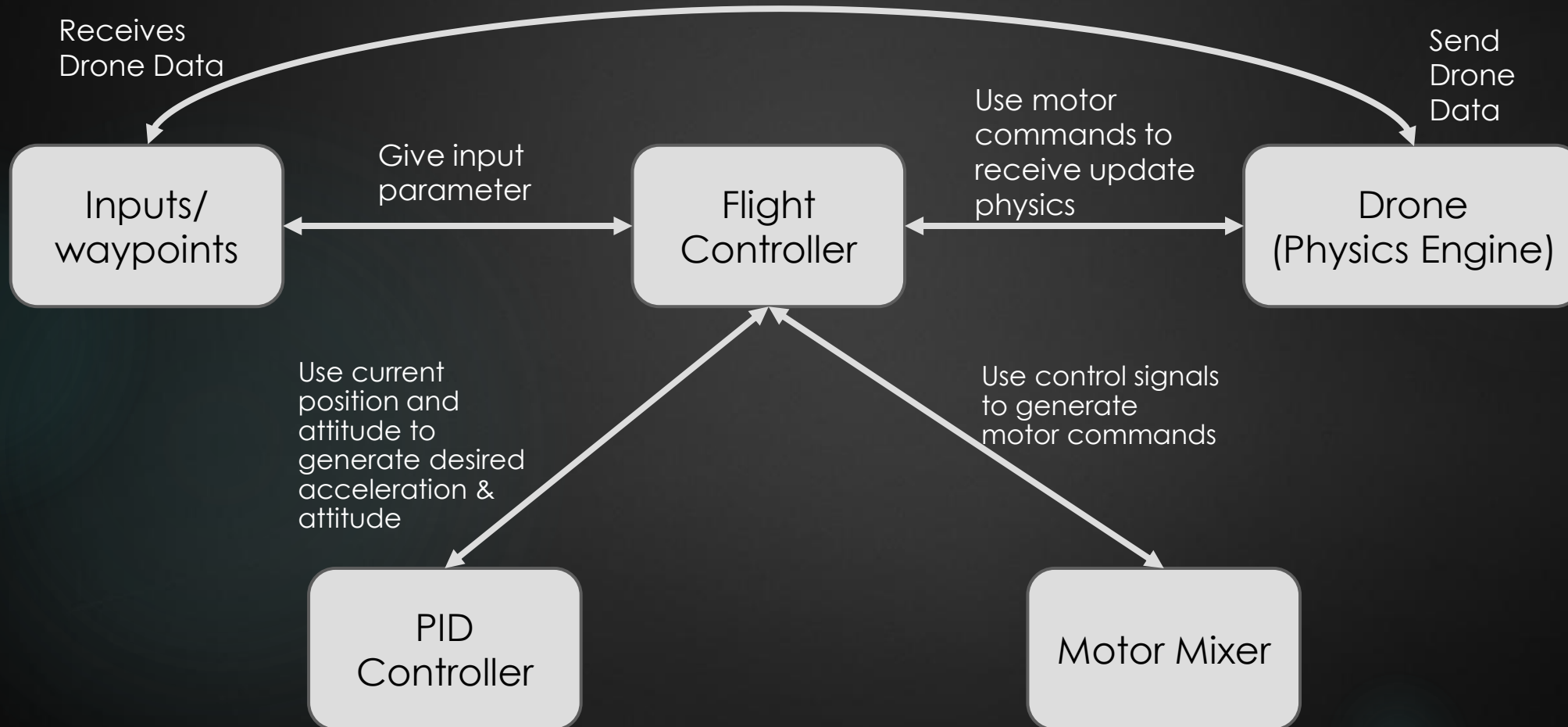
GITHUB LINK: [HTTPS://GITHUB.COM/SALMON490/DRONESIMULATOR](https://github.com/SALMON490/DRONESIMULATOR)

# Motivation

- ▶ **Goal:** Build a quadcopter simulator that is robust enough to handle violent maneuvers and mathematically precise enough for algorithm validation.
- ▶ **Why Build From Scratch?**
  - ▶ Instead of black box physics engine libraries, I understand first principal math and physics in simulation
  - ▶ Want to learn some controls programming and the build a PID controller and flight controller that can fly a quadcopter aggressively

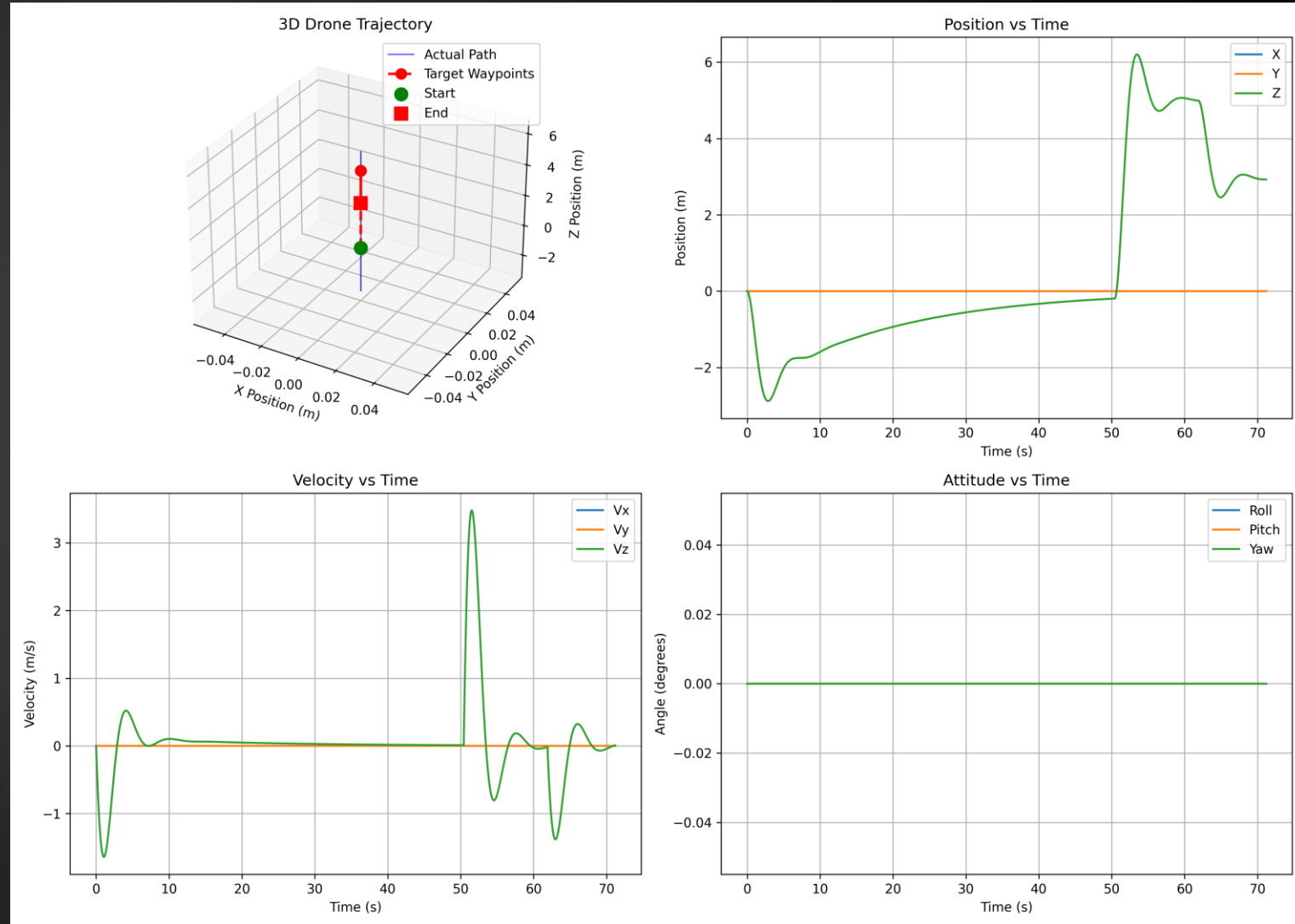


# Overview of Files



# The “Naive” Approach

- ▶ I started by controlling all the motors as a whole so that they produced a total thrust and was using a standard PID and Euler integrations
- ▶ **Result:** This worked for hovering, but I immediately hit a wall when I try to do roll, pitch, and yaw
- ▶ Solution:
  - ▶ Control motors individually
  - ▶ Make PID and error calculation for roll, pitch, and yaw
- ▶ This led to a series of challenges



Drone moving from  $z = 0$ , to  $z = 5$ , to  $z = 3$

# Increasing Complexity

- ▶ To move sideways, I had to control motors individually
  - ▶ Implemented a *MotorMixer* Class using the FLU (Front-Left-Up) convention
  - ▶ Normalized motor outputs to  $[0, 1]$  and applied clamping for values out of bounds.
    - ▶ Led to the **Mixer-Induced Ballooning Problem**
  - ▶ Calculated error of angles with  $\tan^{-1}(\text{accel}/\text{thrust})$ 
    - ▶ Led to the **Small Angle Approximation Problem**

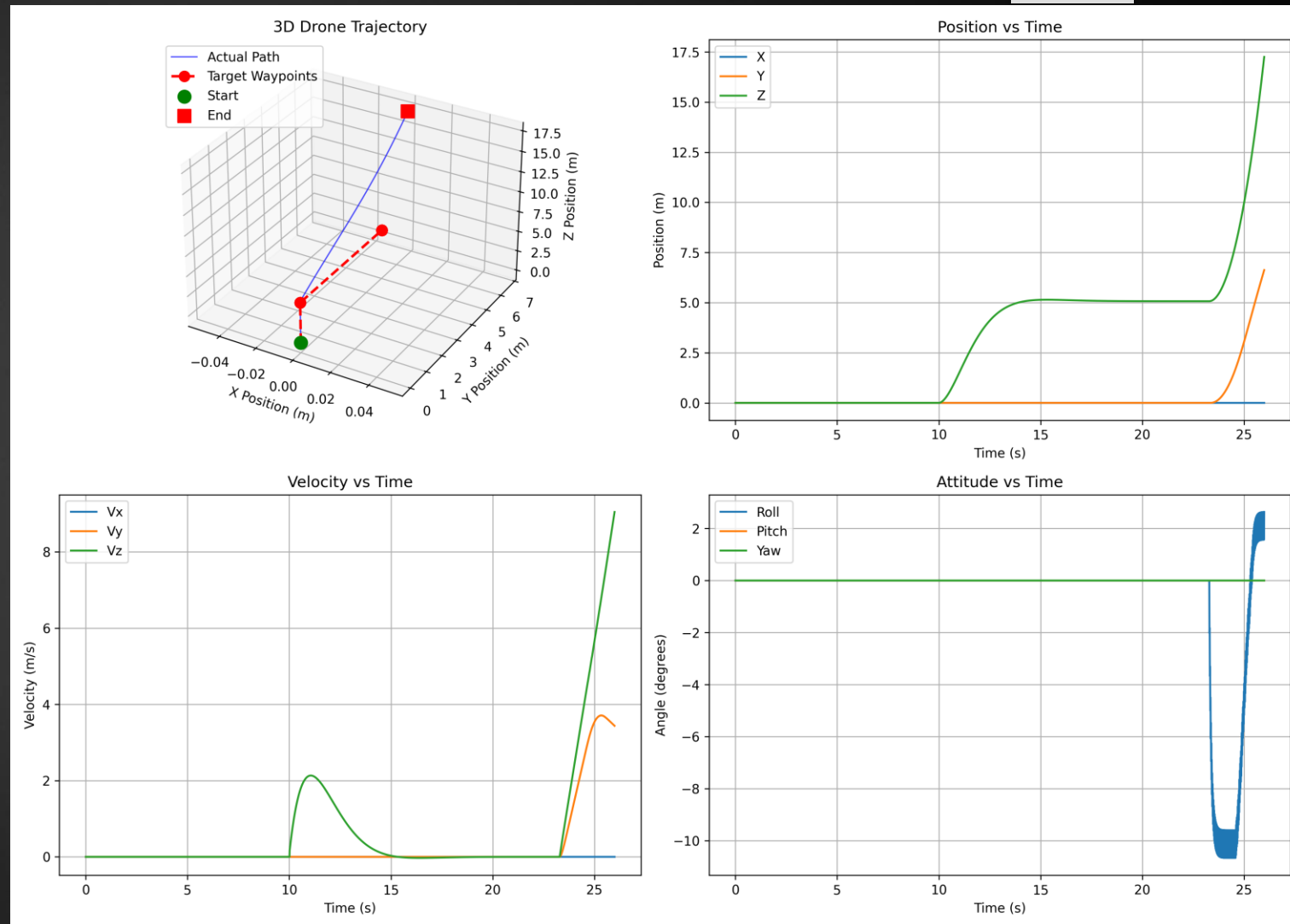
# Mixer-Induced Balloon

- ▶ The initial mixing strategies used a 'floor boost' to handle negative motor commands. This added a net-positive offset to total thrust during hard turns, causing the drone to climb (balloon) unpredictably when saturation occurred."

- ▶ Scenario: During aggressive maneuvers (e.g., high-speed roll), the PID controller requested differential thrust (e.g., -0.4) to a close to zero motor command (e.g., 0.3). This resulted in boosting all motors by 0.1 in motor commands

- ▶ Solution: Headroom-Based Rotational Scaling

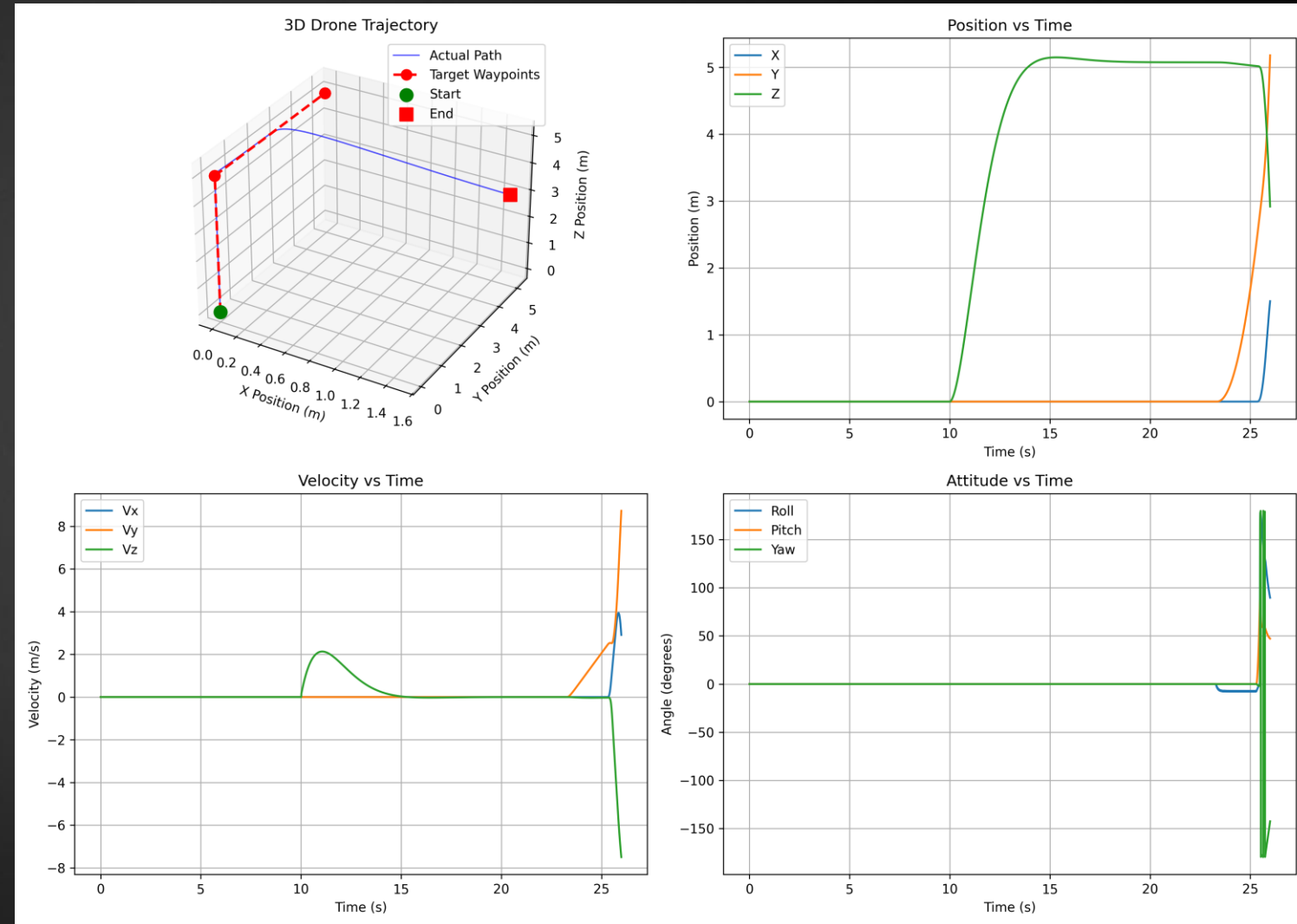
- ▶ Implemented a prioritization algorithm that calculates the available 'headroom' between the current throttle and motor limits. If a turn command exceeds this headroom, the rotational components are scaled down linearly, preserving the base throttle command to guarantee altitude stability.



From (0, 0, 5) to (0, 5, 5), stop simulation before it shot up to space

# Small Angle Approximation

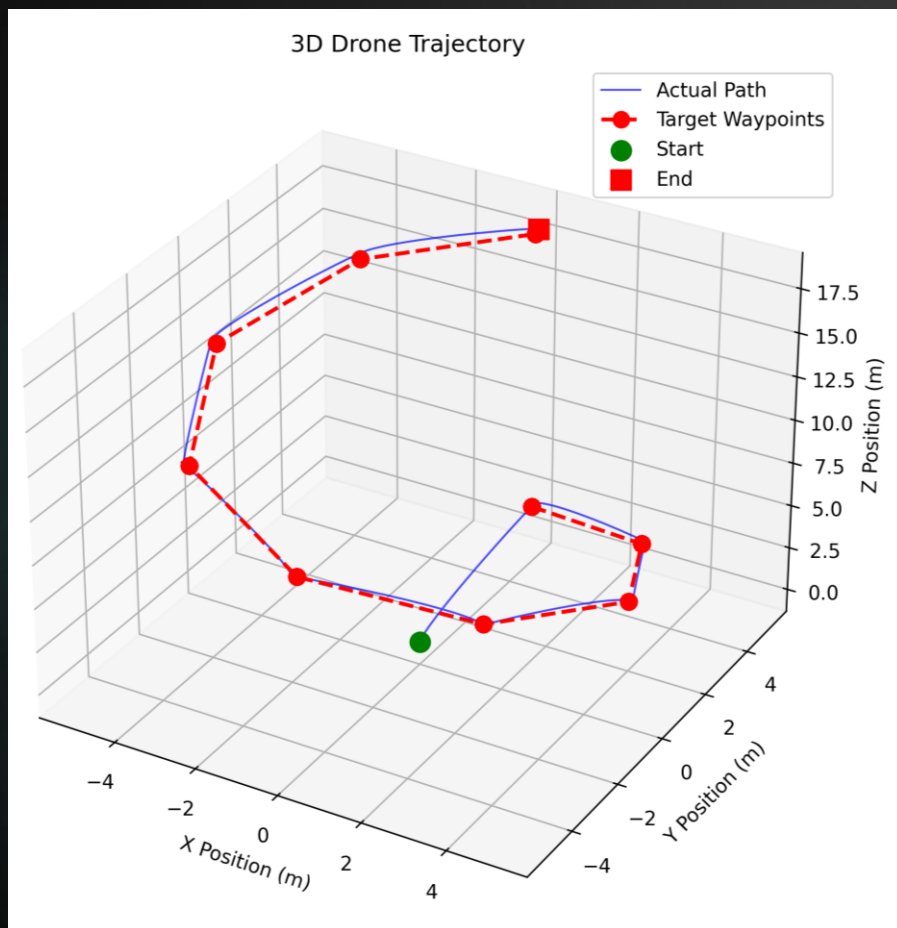
- ▶ Using  $\tan^{-1} \theta$  with acceleration and thrust gave small angle approximation of the desired pitch and roll
  - ▶ if the angle is larger so would the error of the approximation
  - ▶ The code treats X-acceleration as only Pitch and Y-acceleration as only Roll.
- ▶ In reality, when a drone is banked  $45^\circ$ , "Pitch" and "Roll" are coupled.
  - ▶ This simplification causes the drone to wobble or spiral during aggressive diagonal movements.
- ▶ Solution: Thrust Vector Control (TVC)
  - ▶ The controller calculates exactly where the drone should be pushing (Target Z) and compares it to where it is pushing (Current Z) by calculating  $\text{current\_z} \times \text{target\_z}$ . The control loop simply tries to align these two arrows in 3D space. This implicitly handles all axis coupling and naturally provides the correct torque for any orientation.



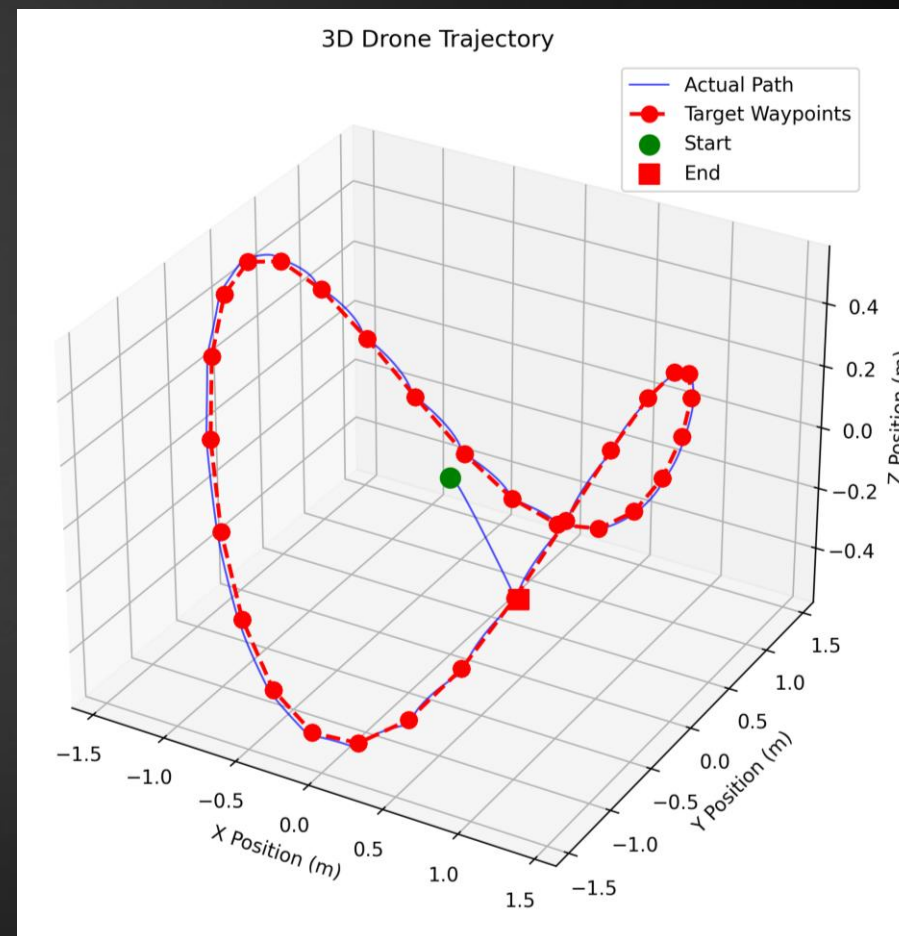
Trying to change to TVC but mixing up signs

# Working Quadcopter Simulation

## Simple Spiral



## Hyperbolic Paraboloid



# Simplifications

## Actuator Dynamics & Power

- **Instant Response:** Motors in-sim represent ideal actuators (Zero lag). Real motors have inertia or inductance, requiring lower PID gains to prevent oscillation.
- **Linear Thrust:** Modeled as  $F \propto \text{Signal}$ . Real thrust is  $F \propto \text{RPM}^2$  with battery voltage sag, requiring battery compensation logic.

## Aerodynamic Model

- **Linear Drag:** Uses  $F = -Cv$ . Real flight follows quadratic drag ( $v^2$ ), meaning the sim underestimates drag at high speeds.
- **Rigid Body:** Frame is perfectly rigid. Real frames vibrate, requiring Notch/Low-Pass filters to prevent noise amplification in the D-Term.

## Perfect Sensors

- **Ground Truth:** Controller gets exact position or orientation. Real drones need an Extended Kalman Filter (EKF) to fuse noisy Gyro/Accel/GPS data.
- **Zero Jitter:** The physics loop is mathematically precise. Embedded hardware suffers from loop jitter and protocol latency (DShot/PWM).

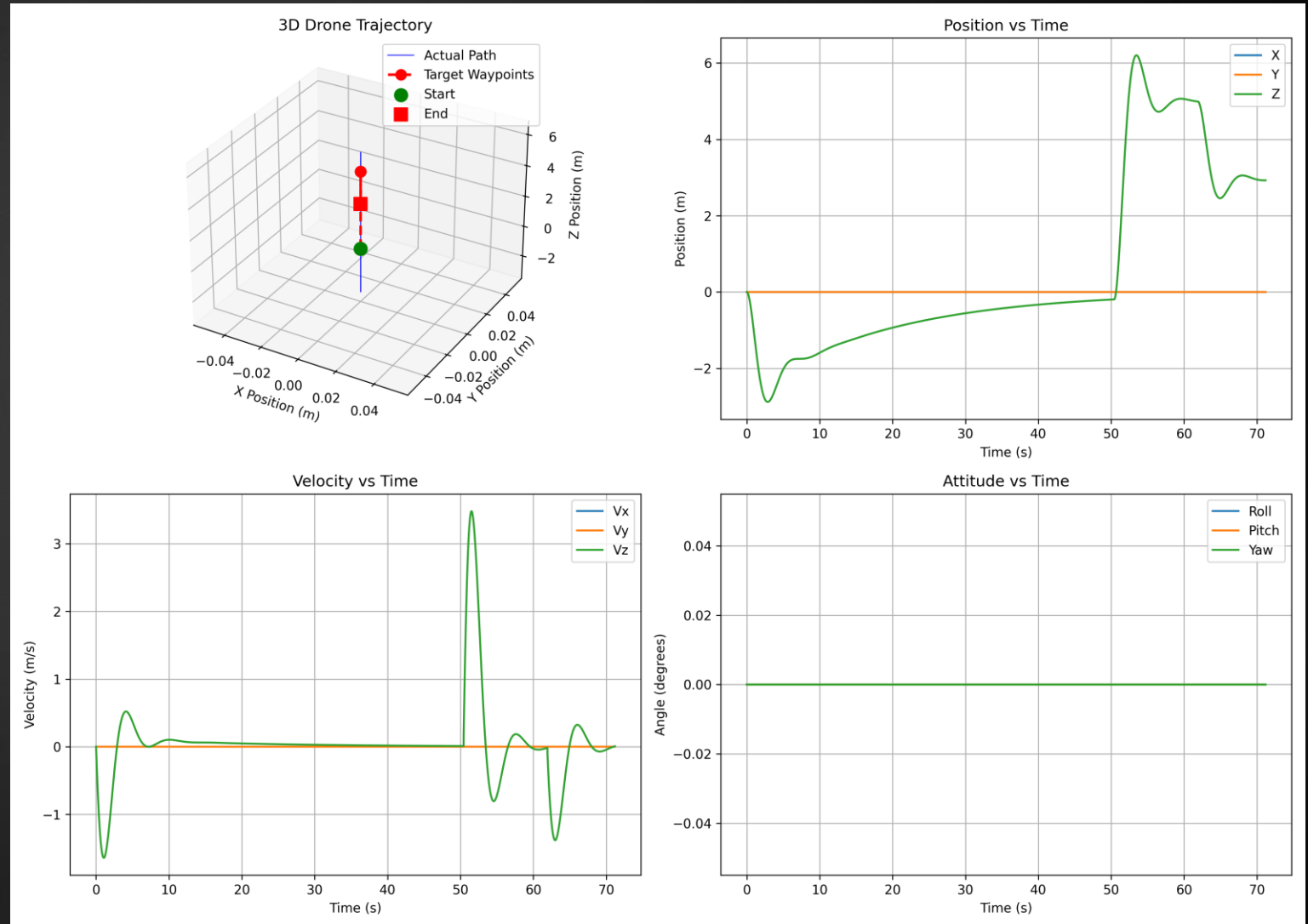
# Future Improvements

- ▶ Port the C++ controller logic to a more realistic simulator like Gazebo and ROS
- ▶ Add multiple sensors with noise
- ▶ Implement Extended Kalman Filter (EKF) for sensor fusion (Simulating noisy IMU data)
- ▶ GitHub Link: <https://github.com/Salmon490/DroneSimulator>

# Graph Explanation

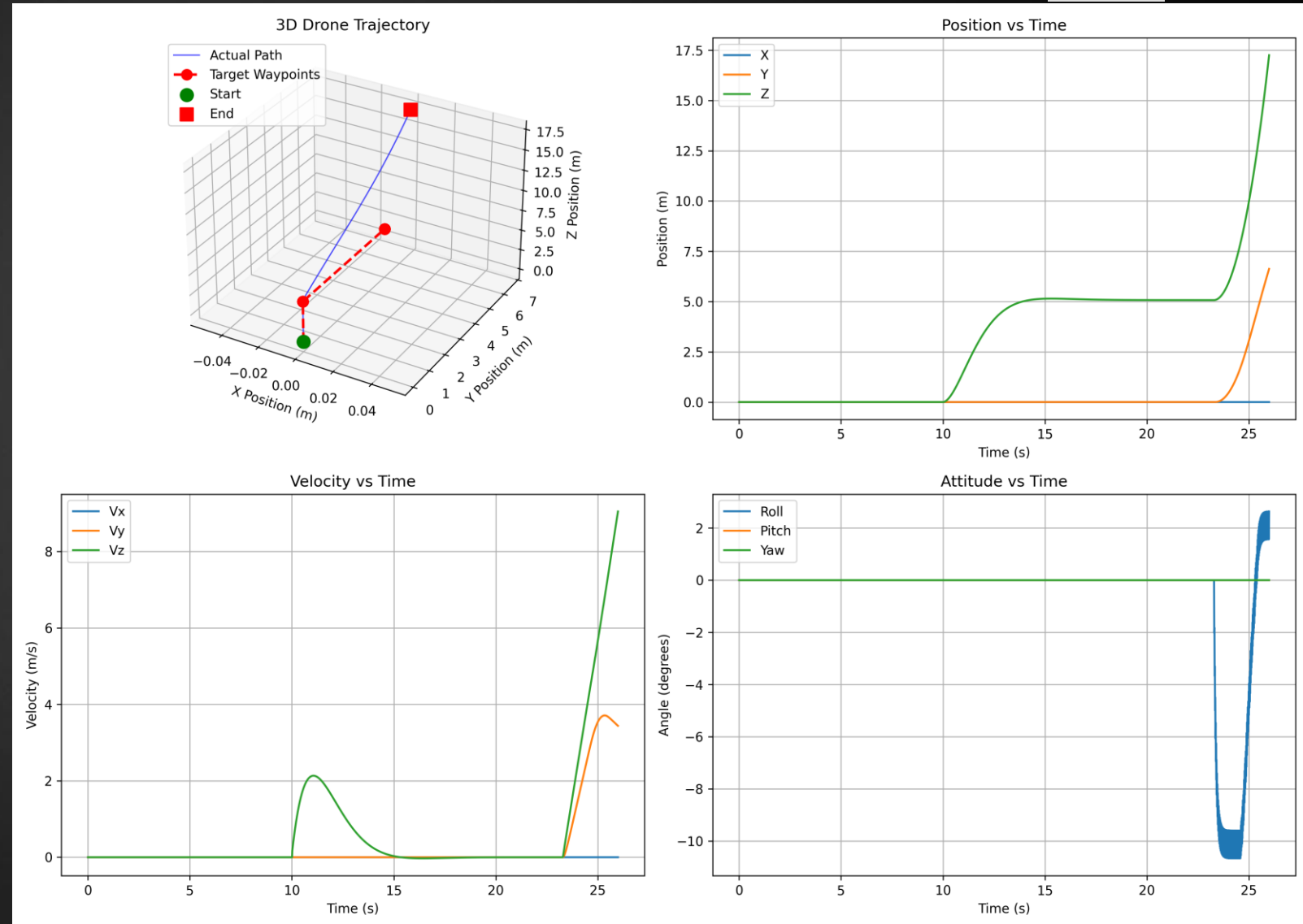
# Controlling all Motors with Average Thrust

- ▶ As you can see, the drone flies up and down pretty well
- ▶ The initial drone in z position is because I have not set the drone to start out with thrust counteracting gravity



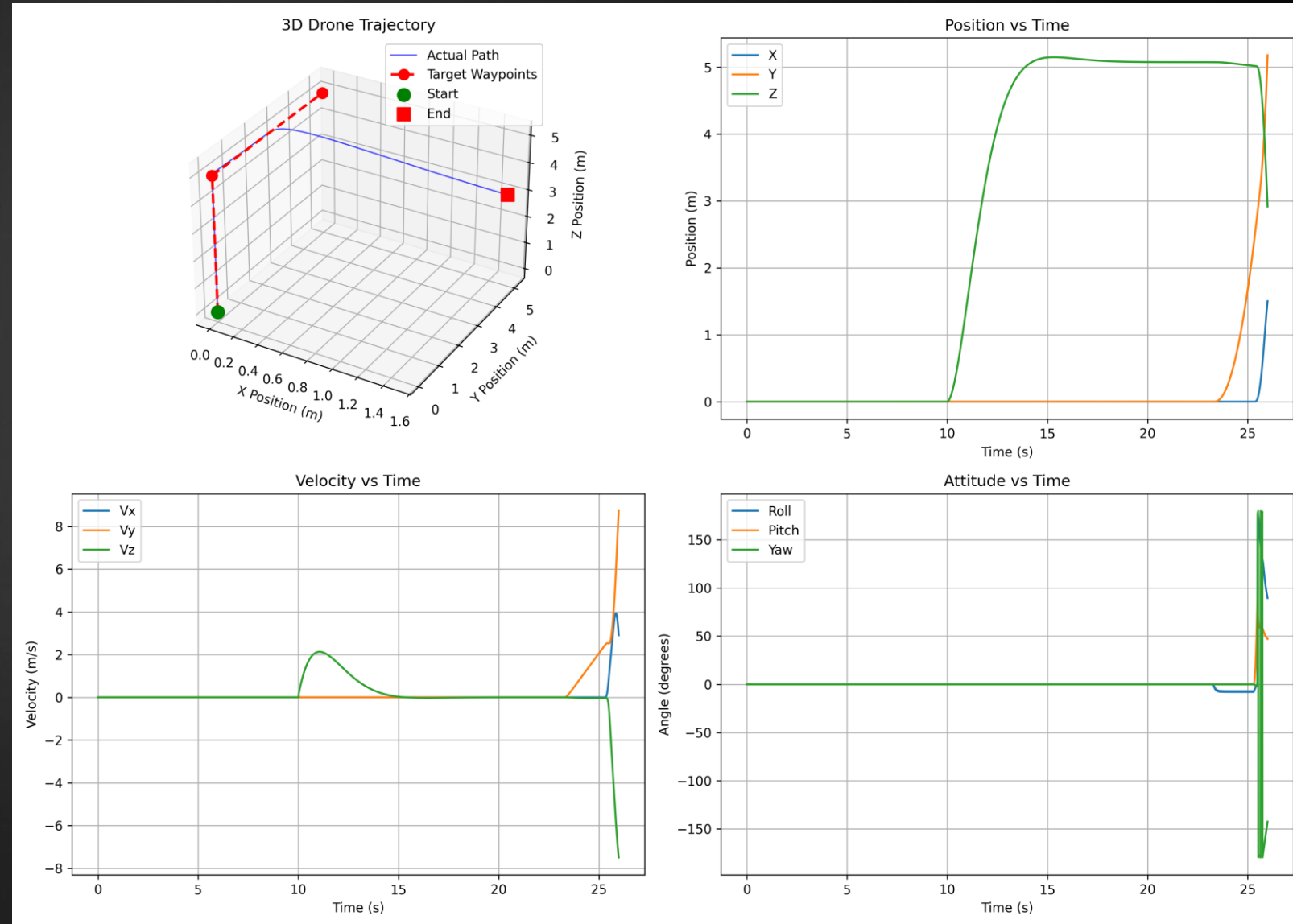
# Trying to go sideways

- The simulation was stop quickly after it uncontrollably flew up
- This was because I did not account for saturated motors and changing them proportionally when motors are saturated then clamped

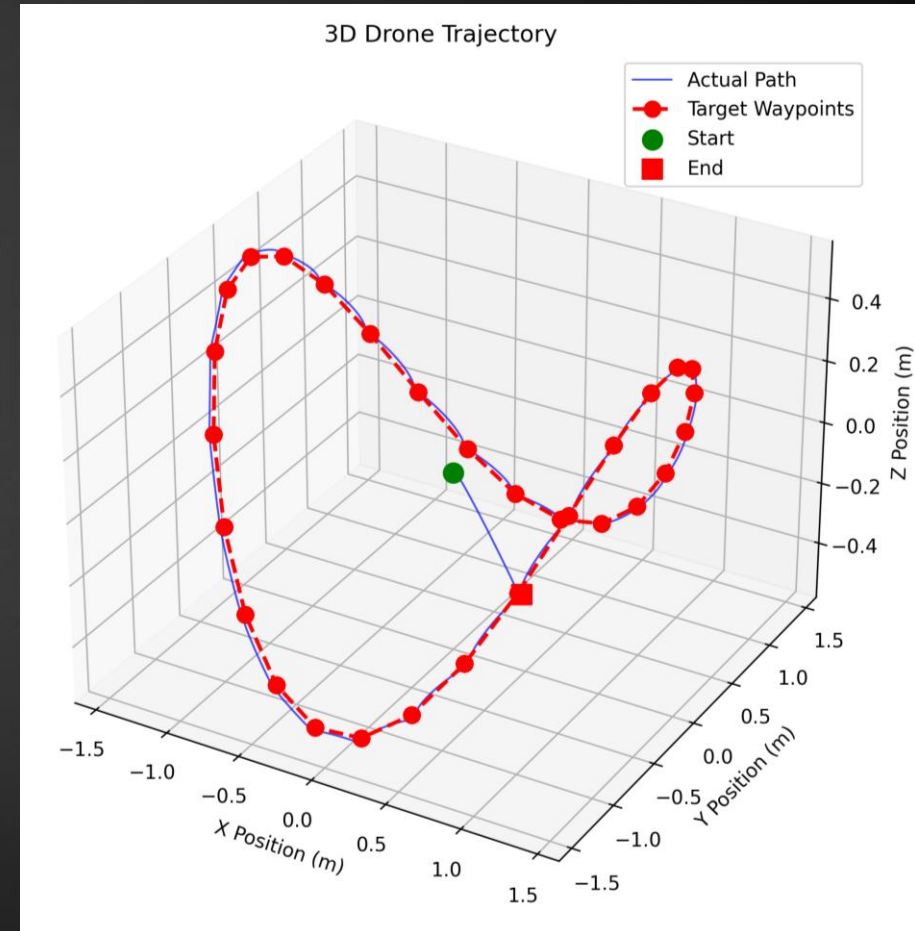
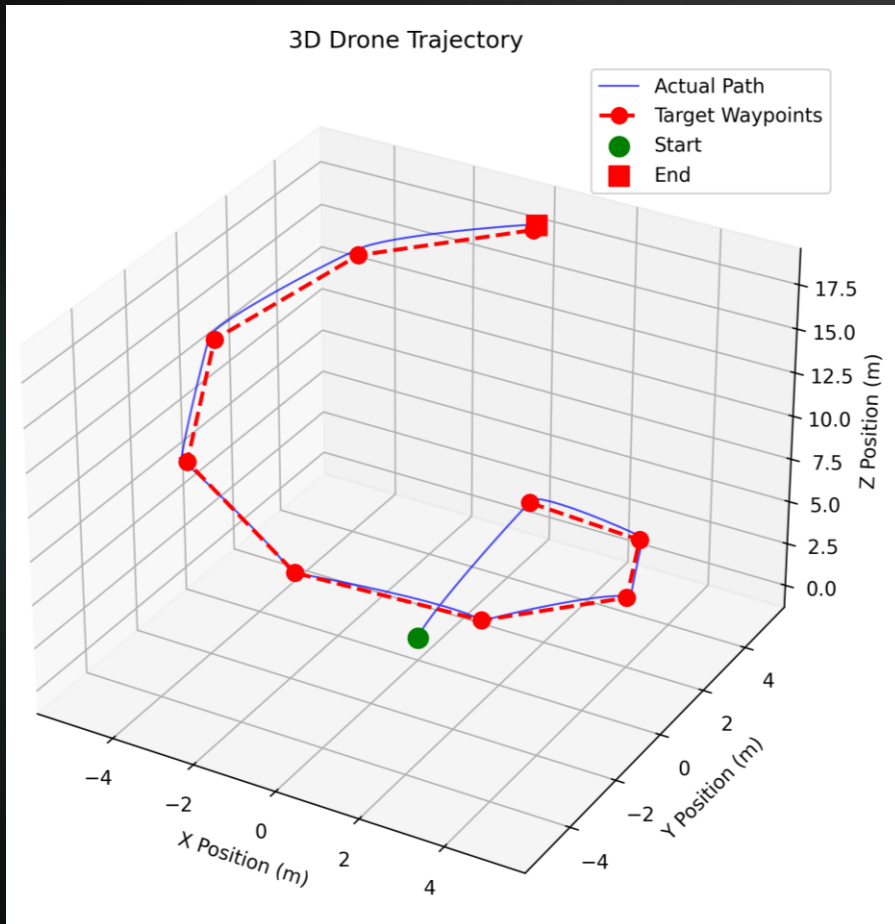


# Coordinate Frame Inconsistency

- ▶ This was result of some research and human error
- ▶ I did not realize some code I found was using a different coordinate system
- ▶ I added some negatives in the wrong places in the code

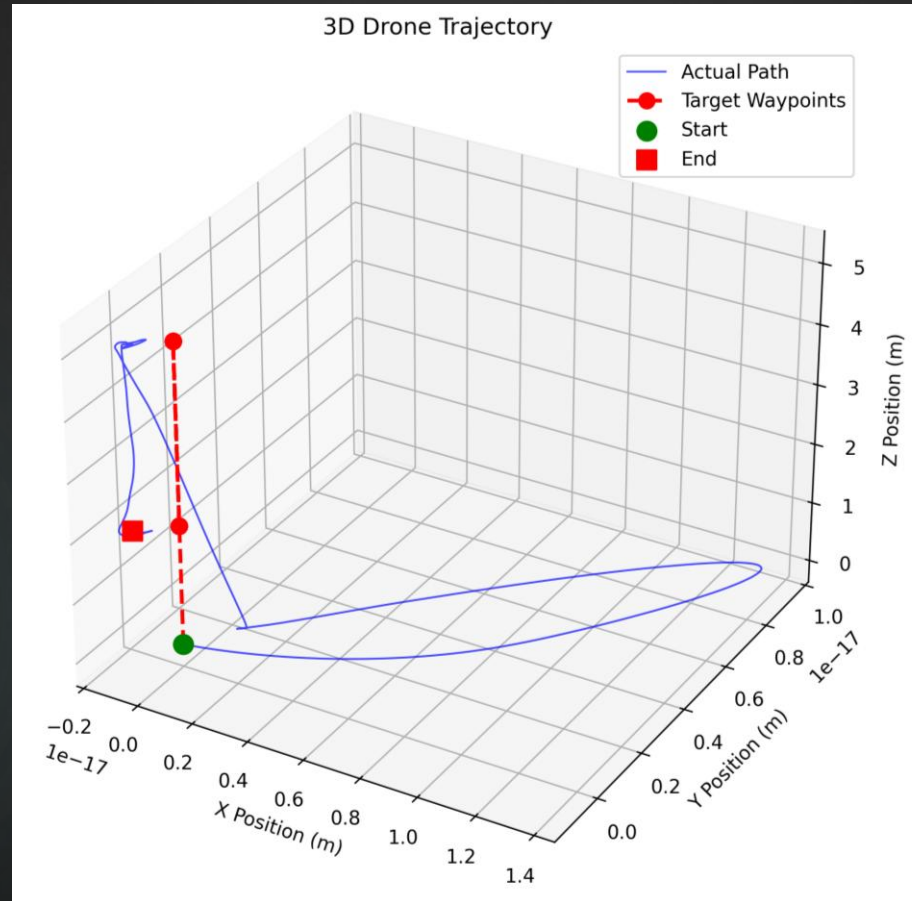


# A working simulation and controls for the quadcopter



# Testing of RK4 (1 of 2)

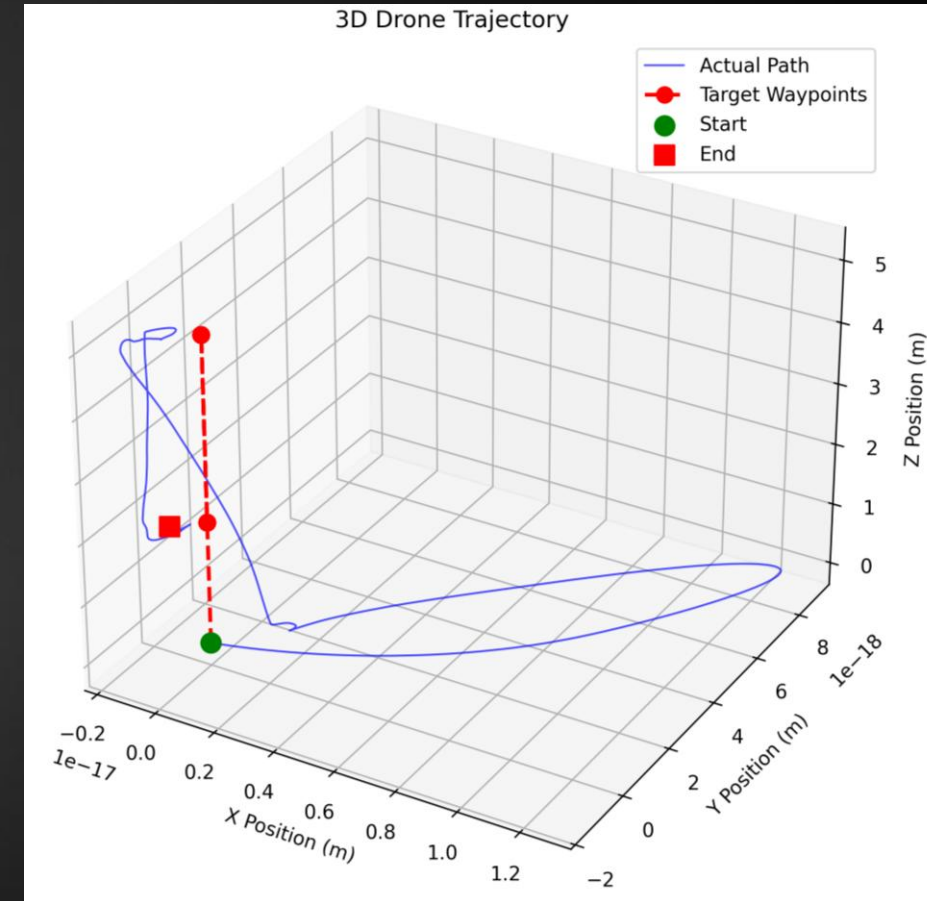
RK4



This was simulated at 0.0001 seconds which is much smaller than it needs to be. But we can take this as the true path that the drone should take

\*Note that the x and y axis are  $1e-17$  meter so these outside of the precision of double

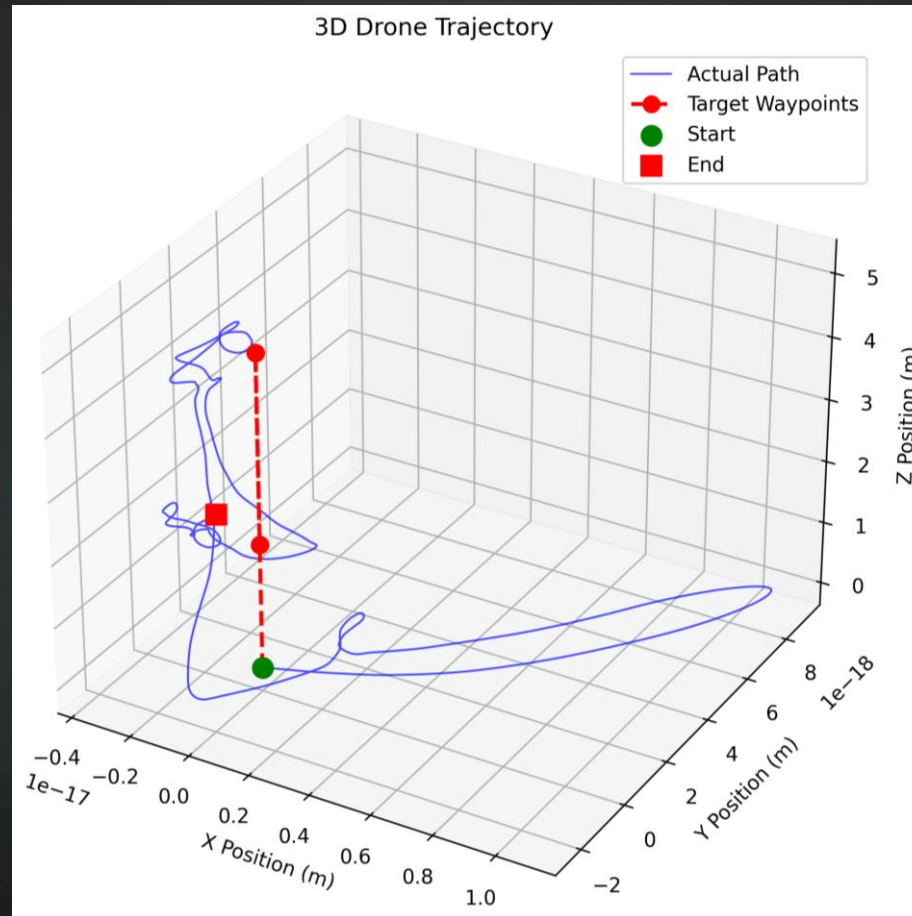
Euler



# Testing RK4 (2 of 2)

This was simulated at 0.002 seconds which is 20 times larger time step and you can see that RK4's path is much closer to the previous graphs

RK4



Euler

